

RAPPORT DE PROJET DE FIN D'ÉTUDES

DIPLÔME UNIVERSITAIRE DE TECHNOLOGIE (DUT) — INFORMATIQUE
DÉCISIONNELLE ET STATISTIQUES

Développement d'un
Chatbot d'Assistance
pour le Site de l'EST
Fkih Ben Salah



Réalisé par : [Votre Prénom et Nom]

Encadrant : [Nom de l'Encadrant Pédagogique]

Année universitaire : 2024 – 2025

*« L'intelligence artificielle ne remplace pas
l'intelligence humaine ; elle l'amplifie. »*

— Fei-Fei Li

Remerciements

Au terme de ce projet de fin d'études, il m'est agréable d'adresser mes sincères remerciements à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce travail.

Je tiens tout d'abord à exprimer ma profonde gratitude à [**Nom de l'Encadrant Pédagogique**], mon encadrant pédagogique, pour son encadrement bienveillant, ses conseils avisés et sa disponibilité tout au long de ce projet. Ses orientations ont été d'une aide précieuse dans la conception et la mise en œuvre de l'application.

Je remercie chaleureusement l'ensemble du corps enseignant et administratif de l'École Supérieure de Technologie de Fkih Ben Salah, et plus particulièrement les professeurs du département **Informatique Décisionnelle et Statistiques**, pour la qualité de la formation dispensée durant ces deux années.

Mes remerciements vont également à la direction de l'EST FBS pour avoir mis à disposition les documents institutionnels nécessaires à la constitution de la base documentaire du chatbot.

Enfin, je remercie profondément ma famille et mes amis pour leur soutien moral indéfectible et leurs encouragements constants qui m'ont permis de mener ce travail à son terme.

Résumé

Ce projet de fin d'études a pour objectif la conception et le développement d'un chatbot d'assistance intelligent destiné au site web de l'École Supérieure de Technologie de Fkih Ben Salah (EST FBS). Face à la difficulté des étudiants et des candidats à trouver rapidement des informations fiables sur les formations, les procédures d'inscription et le règlement intérieur, le projet propose une solution conversationnelle basée sur le paradigme *Retrieval-Augmented Generation* (RAG).

L'architecture mise en place repose sur une pile technologique moderne entièrement en Python : un backend [FastAPI](#) expose les endpoints de l'application, un pipeline RAG orchestré par [LangChain](#) interroge une base vectorielle [Pinecone](#) alimentée par des embeddings [Google Gemini](#). Le modèle de langage [Groq / Llama-3.3-70b-versatile](#) génère les réponses en langage naturel, en s'appuyant strictement sur le contexte documentaire indexé. Chaque interaction est journalisée dans une base de données [PostgreSQL](#) hébergée sur [Supabase](#). L'interface utilisateur est une application mono-page en HTML, CSS et JavaScript pur, au design moderne, avec indicateur de statut en temps réel et affichage des sources citées.

Les résultats montrent la capacité du système à répondre précisément aux questions portant sur les filières, les conditions d'admission et le programme DUT, en citant les sources documentaires utilisées. Les perspectives incluent l'enrichissement de la base documentaire, le déploiement sur un serveur de production et l'ajout d'un mécanisme d'authentification.

Mots-clés : chatbot, RAG, LangChain, Pinecone, FastAPI, Groq, Gemini, intelligence artificielle, traitement du langage naturel, EST FBS.

Abstract

This final-year project aims at designing and developing an intelligent assistance chatbot for the website of the École Supérieure de Technologie de Fkih Ben Salah (EST FBS). Given the difficulty students and applicants face in quickly finding reliable information about programs, enrollment procedures, and institutional regulations, this project proposes a conversational solution based on the *Retrieval-Augmented Generation* (RAG) paradigm.

The implemented architecture relies on a modern Python-based technology stack : a [FastAPI](#) backend exposes the application endpoints, a RAG pipeline orchestrated by [LangChain](#) queries a [Pinecone](#) vector database populated with [Google Gemini](#) embeddings. The [Groq / Llama-3.3-70b-versatile](#) language model generates natural language responses, strictly grounded in the indexed document context. Each interaction is logged in a [PostgreSQL](#) database hosted on [Supabase](#). The user interface is a single-page application built in pure HTML, CSS, and JavaScript, featuring a modern dark-themed design with a real-time status indicator and source citation display.

Results demonstrate the system's ability to accurately answer questions about academic programs, admission requirements, and the DUT curriculum, while citing the referenced source documents. Future perspectives include enriching the document corpus, deploying to a production server, and adding an authentication mechanism.

Keywords : chatbot, RAG, LangChain, Pinecone, FastAPI, Groq, Gemini, artificial intelligence, natural language processing, EST FBS.

Table des matières

Remerciements	5
Résumé	7
Abstract	9
Liste des figures	15
Liste des tableaux	17
Liste des abréviations	19
Introduction générale	21
1 Contexte général du projet	23
1.1 Présentation de l'EST Fkih Ben Salah	23
1.1.1 Mission et valeurs	23
1.1.2 Offre de formation	23
1.2 Contexte du projet	24
1.3 Problématique	24
1.4 Objectifs du projet	25
1.5 Méthodologie adoptée	25
2 Analyse et spécification des besoins	27
2.1 Étude de l'existant	27
2.1.1 Site institutionnel de l'EST FBS	27
2.1.2 Solutions chatbot existantes	27
2.2 Analyse des besoins fonctionnels	27
2.3 Analyse des besoins non fonctionnels	28
2.4 Acteurs du système	28
2.5 Cas d'utilisation	29
2.5.1 Diagramme de cas d'utilisation	29
2.5.2 Cahier des charges fonctionnel	29
3 Conception et architecture du système	31
3.1 Architecture globale	31
3.2 Architecture logicielle	31
3.2.1 Pipeline RAG	31

3.3	Diagramme de classes	32
3.4	Diagramme de séquence	33
3.5	Modèle de données	33
3.6	Description des composants	34
4	Réalisation et développement	35
4.1	Environnement de développement	35
4.1.1	Environnement matériel	35
4.1.2	Environnement logiciel	35
4.2	Technologies utilisées	36
4.2.1	Python et FastAPI	36
4.2.2	LangChain et le pipeline RAG	36
4.2.3	Pinecone — Base vectorielle	36
4.2.4	Google Gemini Embeddings	37
4.2.5	Groq — LLM d'inférence	37
4.2.6	Supabase PostgreSQL	37
4.3	Structure du projet	37
4.4	Description des modules développés	38
4.4.1	Module API — <code>app/main.py</code>	38
4.4.2	Module RAG — <code>app/rag.py</code>	38
4.4.3	Module Base de données — <code>app/database.py</code>	39
4.4.4	Script d'indexation — <code>scripts/load_vectors.py</code>	39
4.4.5	Frontend — <code>index.html</code>	39
4.5	Implémentation du chatbot	39
4.5.1	Flux de traitement d'une question	40
4.6	Gestion des données	40
4.6.1	Documents indexés	40
4.7	Difficultés rencontrées et solutions apportées	41
5	Tests, limites et perspectives	43
5.1	Stratégie de test	43
5.2	Tests fonctionnels	43
5.3	Analyse des performances	44
5.4	Limites du système	44
5.5	Perspectives d'amélioration	45
	Conclusion générale	47
	Bibliographie	49
A	Variables d'environnement	51
B	Guide d'installation et de démarrage	52

C Endpoints de l'API — Documentation Swagger	53
D Diagrammes PlantUML	54

Table des figures

2.1	Diagramme de cas d'utilisation du chatbot EST FBS	29
3.1	Architecture globale du système chatbot	31
3.2	Pipeline RAG du chatbot EST FBS	32
3.3	Diagramme de classes principal	32
3.4	Diagramme de séquence — Flux principal de traitement d'une question	33

Liste des tableaux

1.1	Offre de formation de l'EST FBS	24
1.2	Objectifs du projet	25
2.1	Comparaison des approches de chatbot	27
2.2	Besoins fonctionnels	28
2.3	Besoins non fonctionnels	28
2.4	Cahier des charges fonctionnel	29
3.1	Structure de la table <code>chat_logs</code>	33
3.2	Description des composants de l'application	34
4.1	Environnement matériel de développement	35
4.2	Environnement logiciel utilisé	35
4.3	Paramètres du découpage textuel	36
4.4	Paramètres du LLM Groq	37
4.5	Documents du corpus documentaire	40
4.6	Difficultés et solutions	41
5.1	Résultats des tests fonctionnels	43
5.2	Indicateurs de performance	44
5.3	Pistes d'amélioration futures	45
C.1	Endpoints de l'API FastAPI	53

Liste des abréviations

Abréviation	Signification
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DUT	Diplôme Universitaire de Technologie
ENV	Environment Variable
EST	École Supérieure de Technologie
EST FBS	École Supérieure de Technologie de Fkih Ben Salah
FBS	Fkih Ben Salah
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IA	Intelligence Artificielle
IDS	Informatique Décisionnelle et Statistiques
IHM	Interface Homme-Machine
JS	JavaScript
JSON	JavaScript Object Notation
LLM	Large Language Model
NLP	Natural Language Processing
ORM	Object-Relational Mapping
PDF	Portable Document Format
PFE	Projet de Fin d'Études
PgSQL	PostgreSQL
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SPA	Single-Page Application
SQL	Structured Query Language
SVG	Scalable Vector Graphics
TXT	Text (format fichier)
URL	Uniform Resource Locator
USMS	Université Sultan Moulay Slimane

Abréviation	Signification
UUID	Universally Unique Identifier
WSGI	Web Server Gateway Interface

Introduction générale

L'essor fulgurant de l'intelligence artificielle et du traitement automatique du langage naturel a profondément transformé la manière dont les organisations interagissent avec leurs utilisateurs. Les agents conversationnels, ou chatbots, sont devenus des outils incontournables dans de nombreux secteurs, offrant une disponibilité permanente, une réactivité immédiate et la capacité de traiter un grand volume de requêtes simultanément.

Dans le contexte de l'enseignement supérieur, les établissements font face à un défi récurrent : les étudiants et candidats peinent souvent à trouver rapidement et de manière autonome les informations dont ils ont besoin. Que ce soit pour connaître les conditions d'admission, comprendre le programme pédagogique ou accéder au règlement intérieur, les démarches sont parfois fastidieuses et chronophages.

L'École Supérieure de Technologie de Fkih Ben Salah (EST FBS), composante de l'Université Sultan Moulay Slimane, n'échappe pas à cette problématique. Bien que son site web institutionnel fournisse des informations essentielles, celles-ci restent difficiles d'accès, dispersées et non interactives.

C'est dans ce cadre que s'inscrit ce Projet de Fin d'Études, réalisé au titre du Diplôme Universitaire de Technologie en Informatique Décisionnelle et Statistiques. Il consiste à concevoir et développer un chatbot d'assistance intelligent basé sur l'architecture *Retrieval-Augmented Generation* (RAG), capable de répondre aux questions des utilisateurs en s'appuyant sur les documents officiels de l'école.

Le présent rapport est structuré en cinq chapitres complémentaires :

- Le **chapitre 1** présente le contexte général du projet, la problématique identifiée et les objectifs visés.
- Le **chapitre 2** développe l'analyse et la spécification des besoins fonctionnels et non fonctionnels, les acteurs et les cas d'utilisation.
- Le **chapitre 3** expose la conception architecturale du système, les diagrammes UML et le modèle de données.
- Le **chapitre 4** détaille les phases de réalisation et de développement, les technologies employées et les modules implémentés.

- Le **chapitre 5** présente la stratégie de test, les résultats, les limites du système et les perspectives d'amélioration.

Une conclusion générale synthétise les apports du projet et ouvre des pistes pour de futurs développements.

Contexte général du projet

1.1 Présentation de l'EST Fkih Ben Salah

L'École Supérieure de Technologie de Fkih Ben Salah (EST FBS) est un établissement public d'enseignement supérieur créé par décision du Conseil du Gouvernement en date du **11 avril 2019**. Elle constitue une composante de l'Université Sultan Moulay Slimane (USMS) et a accueilli ses premiers étudiants en **septembre 2019**.

Implantée sur le Hay Tighnari, Route nationale N11 de Casablanca, commune de Fkih Ben Salah (Boite Postale 336), l'EST FBS a pour vocation de proposer des formations à finalité professionnelle directement articulées avec les besoins du tissu économique régional et national.

1.1.1 Mission et valeurs

L'EST FBS poursuit quatre missions fondamentales :

1. Offrir des formations en adéquation avec le milieu professionnel et le marché du travail.
2. Renouveler fréquemment les filières et enrichir les contenus pédagogiques.
3. Conduire des activités de recherche et d'innovation technologique en partenariat avec le secteur industriel.
4. Promouvoir la coopération régionale, nationale et internationale.

1.1.2 Offre de formation

L'établissement propose deux niveaux de diplômes, résumés dans le tableau 1.1.

Table 1.1 – Offre de formation de l'EST FBS

Diplôme	Filières
DUT (Bac+2)	Génie Civil, Génie Minier, Génie Informatique, Génie Biologique, Industrie Agroalimentaire, Agrochimie, Informatique Décisionnelle et Statistiques , Géométallurgie et Géotechnique Minière, Chimie & Techniques d'Analyse
Licence Professionnelle (Bac+3)	Techniques d'Irrigation et Énergies Renouvelables, Biotechnologies Agroalimentaires, Génie Civil, Informatique Décisionnelle et Statistiques, Sciences Agronomiques

1.2 Contexte du projet

Dans le cadre du module de Projet de Fin d'Études de la deuxième année de DUT, filière Informatique Décisionnelle et Statistiques, ce projet est proposé en réponse à un besoin réel exprimé par l'institution : améliorer l'accès à l'information pour les étudiants et les candidats via une solution numérique innovante.

L'augmentation du nombre d'étudiants inscrits à l'EST FBS, couplée à la diversification des filières proposées, génère un flux croissant de demandes d'informations répétitives adressées à l'administration. Une solution automatisée permettrait de décharger le personnel administratif tout en offrant une meilleure expérience utilisateur.

1.3 Problématique

L'analyse de la situation existante révèle plusieurs problèmes concrets :

Problèmes identifiés

- Les informations sur les formations, les admissions et le règlement sont dispersées sur plusieurs pages du site institutionnel.
- Le site actuel n'offre aucune interface conversationnelle permettant des requêtes en langage naturel.
- L'administration reçoit un volume important de questions redondantes pouvant être automatiquement traitées.
- Les horaires d'ouverture de l'administration limitent la disponibilité du support étudiant.
- Aucun outil de traçabilité des questions posées n'existe à ce jour.

La problématique centrale peut donc se formuler ainsi : **Comment mettre à disposition des utilisateurs du site de l’EST FBS un assistant conversationnel capable de répondre précisément, en langage naturel et en temps réel, à leurs questions, en s’appuyant sur les documents officiels de l’établissement ?**

1.4 Objectifs du projet

Ce projet poursuit les objectifs suivants :

Table 1.2 – *Objectifs du projet*

#	Objectif	Description
1	Objectif principal	Développer un chatbot RAG fonctionnel répondant aux questions sur l’EST FBS
2	Indexation documentaire	Vectoriser les documents PDF et TXT officiels dans une base vectorielle
3	Interface web	Concevoir une IHM moderne, accessible et intuitive
4	API REST	Exposer le moteur RAG via une API FastAPI structurée
5	Journalisation	Enregistrer les interactions dans une base de données PostgreSQL
6	Maintenabilité	Permettre l’ajout aisé de nouveaux documents sans modifier le code

1.5 Méthodologie adoptée

Le projet a été développé selon une approche *itérative et incrémentale*, proche du cadre agile, qui permet une adaptation progressive aux retours d’expérience. Les grandes phases sont les suivantes :

1. **Phase d’analyse** : étude de l’existant, recueil des besoins, identification des acteurs.
2. **Phase de conception** : définition de l’architecture RAG, modélisation UML.
3. **Phase de développement** : implémentation du pipeline RAG, de l’API et du frontend.
4. **Phase d’indexation** : préparation et vectorisation des documents officiels.
5. **Phase de test** : validation fonctionnelle et tests de robustesse.
6. **Phase de documentation** : rédaction du rapport et des guides d’utilisation.

Analyse et spécification des besoins

2.1 Étude de l'existant

Avant d'entamer la conception du chatbot, une analyse de l'existant a été menée afin d'identifier les solutions déjà en place et leurs limites.

2.1.1 Site institutionnel de l'EST FBS

Le site officiel <https://estfbs.usms.ac.ma> présente les informations suivantes : présentation de l'école, offre de formation, actualités et contacts. Cependant, cette solution statique souffre de plusieurs limitations : absence d'interactivité, navigation non contextuelle et absence de moteur de recherche interne efficace.

2.1.2 Solutions chatbot existantes

Deux grandes catégories de chatbots existent dans le domaine académique :

Table 2.1 – Comparaison des approches de chatbot

Critère	Chatbot basé sur règles	Chatbot RAG (retenu)
Flexibilité	Faible	Élevée
Mise à jour	Manuelle et coûteuse	Réindexation automatique
Précision	Limitée aux scénarios prévus	Basée sur les documents réels
Maintenance	Complexe	Simple (ajout de fichiers)
Coût de déploiement	Variable	Modèle cloud SaaS

L'architecture RAG a été retenue car elle permet de lier les réponses générées aux documents sources officiels, assurant ainsi fiabilité et traçabilité.

2.2 Analyse des besoins fonctionnels

Les besoins fonctionnels représentent les fonctionnalités que le système doit fournir :

Table 2.2 – *Besoins fonctionnels*

ID	Fonctionnalité	Description
BF-01	Poser une question	L'utilisateur saisit une question en langage naturel
BF-02	Obtenir une réponse	Le système retourne une réponse contextuelle générée par le LLM
BF-03	Afficher les sources	Les documents utilisés pour générer la réponse sont cités
BF-04	Questions suggérées	Des raccourcis thématiques orientent l'utilisateur
BF-05	Vérifier le statut API	L'interface indique si le backend est disponible
BF-06	Historique conversationnel	Les 6 derniers échanges sont transmis au RAG pour maintenir le contexte
BF-07	Indexer des documents	Un script permet d'ajouter de nouveaux PDF/TXT à la base vectorielle
BF-08	Journaliser les interactions	Chaque question-réponse est enregistrée en base de données

2.3 Analyse des besoins non fonctionnels

Table 2.3 – *Besoins non fonctionnels*

ID	Propriété	Description
BNF-01	Performance	Temps de réponse du RAG inférieur à 10 secondes (timeout configuré à 30 s)
BNF-02	Disponibilité	Service accessible 24h/24 après déploiement
BNF-03	Sécurité	Clés API stockées dans des variables d'environnement non commitées
BNF-04	Maintenabilité	Ajout de document par simple copie dans <code>data/</code> + réindexation
BNF-05	Portabilité	Application conteneurisable, indépendante du système d'exploitation
BNF-06	Extensibilité	Architecture modulaire permettant le remplacement des composants
BNF-07	Fiabilité	Réponses fondées exclusivement sur le corpus documentaire officiel
BNF-08	Convivialité	Interface intuitive en français, support multilingue affiché (FR/EN/AR)

2.4 Acteurs du système

Le système identifie deux acteurs principaux :

- **Utilisateur (étudiant / candidat)** : interagit avec le chatbot via l’interface web pour poser des questions sur l’EST FBS.
- **Administrateur** : gère la base documentaire (ajout, suppression) et lance le script d’indexation. Il surveille également les journaux d’interactions via Supabase.

2.5 Cas d’utilisation

2.5.1 Diagramme de cas d’utilisation

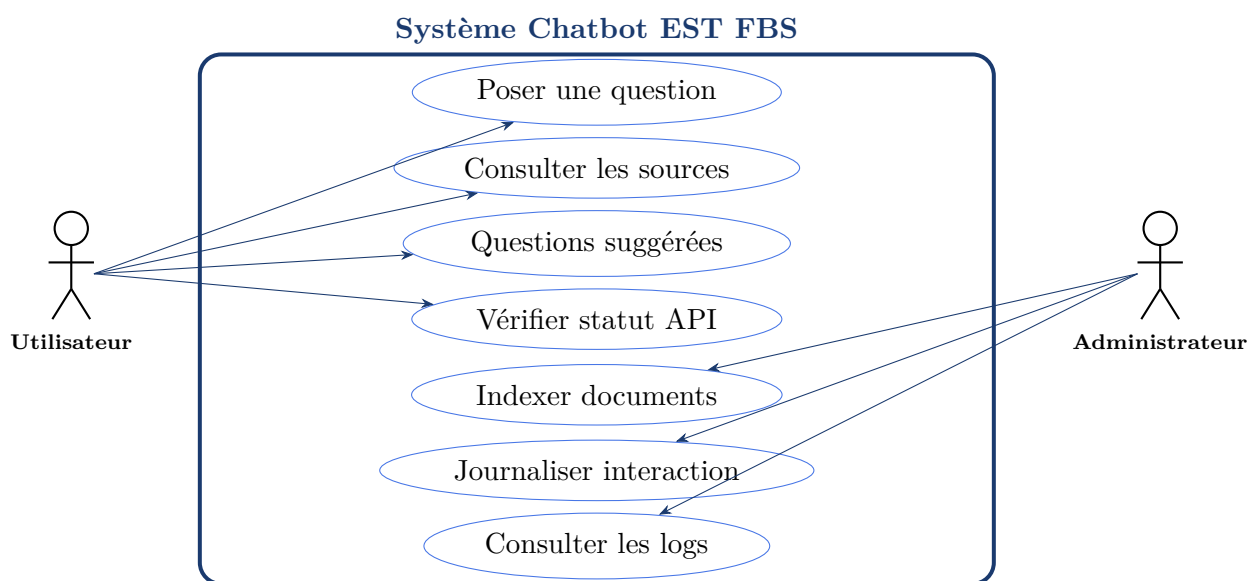


Figure 2.1 – Diagramme de cas d’utilisation du chatbot EST FBS

2.5.2 Cahier des charges fonctionnel

Table 2.4 – Cahier des charges fonctionnel

Fonction	Critère	Mesure	Priorité
Réponse pertinente	Précision	Basée sur top-3 chunks pertinents	Critique
Temps de réponse	Performance	< 10 secondes	Haute
Affichage sources	Traçabilité	Nom du fichier source affiché	Haute
Historique	Contexte	6 derniers tours conservés	Moyenne
Disponibilité	Uptime	Endpoint <code>/health</code> opérationnel	Haute
Journalisation	Persistance	Sauvegarde dans <code>chat_logs</code>	Haute
Indexation	Mise à jour	Script CLI en une commande	Moyenne

Conception et architecture du système

3.1 Architecture globale

Le système repose sur une architecture trois-tiers augmentée d'un pipeline RAG. La figure 3.1 illustre les flux entre les composants.

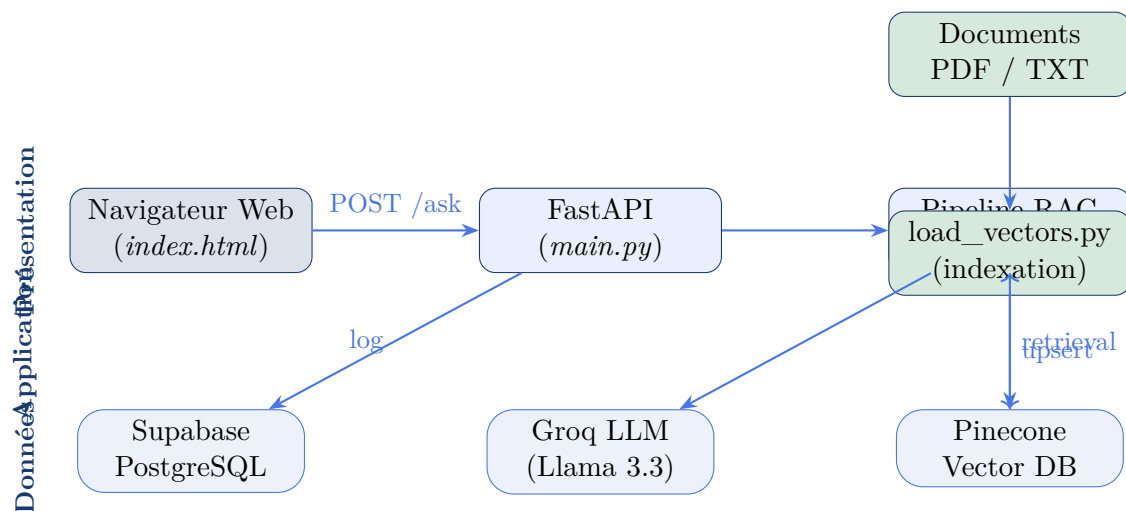


Figure 3.1 – Architecture globale du système chatbot

3.2 Architecture logicielle

3.2.1 Pipeline RAG

Le pipeline *Retrieval-Augmented Generation* suit les étapes décrites ci-dessous et illustrées en figure 3.2 :

1. L'utilisateur soumet une question via l'interface web.
2. La question est transmise à **FastAPI** via un appel `POST /ask`.
3. Le service RAG (`rag.py`) vectorise la question avec les embeddings Gemini.

4. Les $k = 3$ chunks les plus proches sémantiquement sont récupérés depuis Pinecone.
5. Le contexte documentaire et l'historique conversationnel (6 tours max) sont injectés dans un prompt structuré.
6. Le LLM Groq (Llama-3.3-70b-versatile) génère la réponse finale.
7. La réponse, accompagnée des noms de sources, est retournée au frontend et logguée dans Supabase.

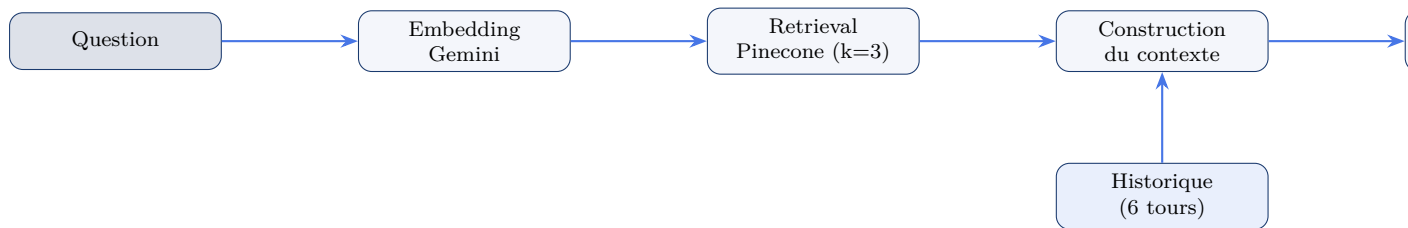


Figure 3.2 – Pipeline RAG du chatbot EST FBS

3.3 Diagramme de classes

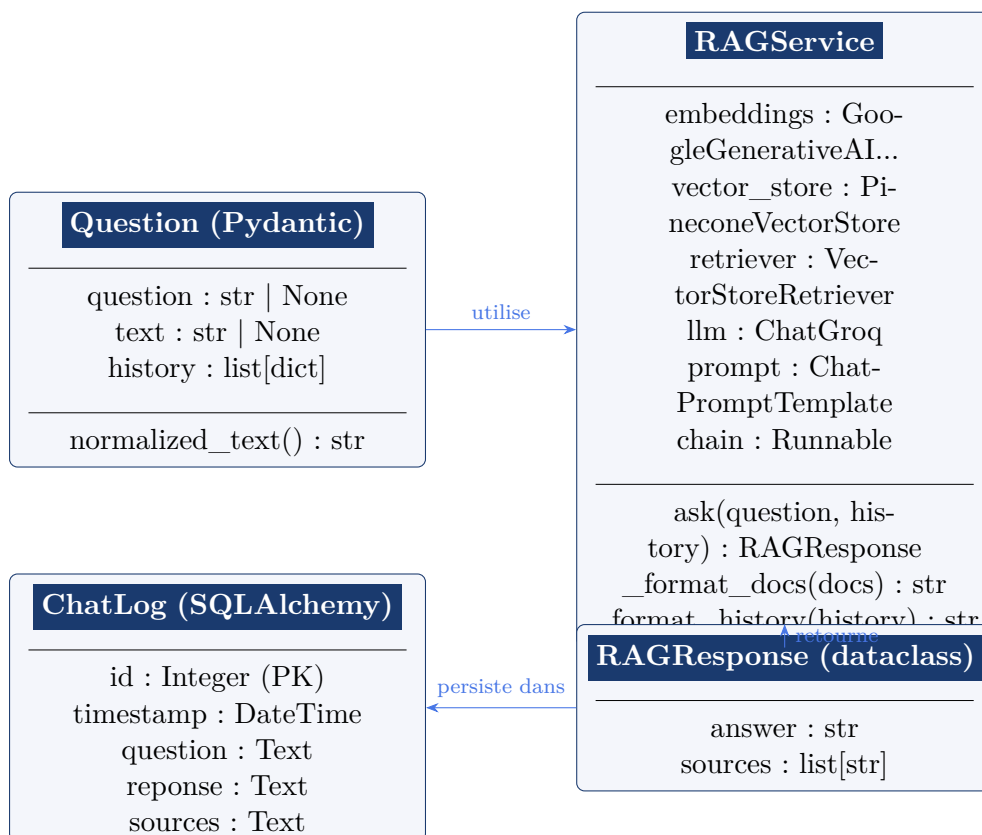


Figure 3.3 – Diagramme de classes principal

3.4 Diagramme de séquence

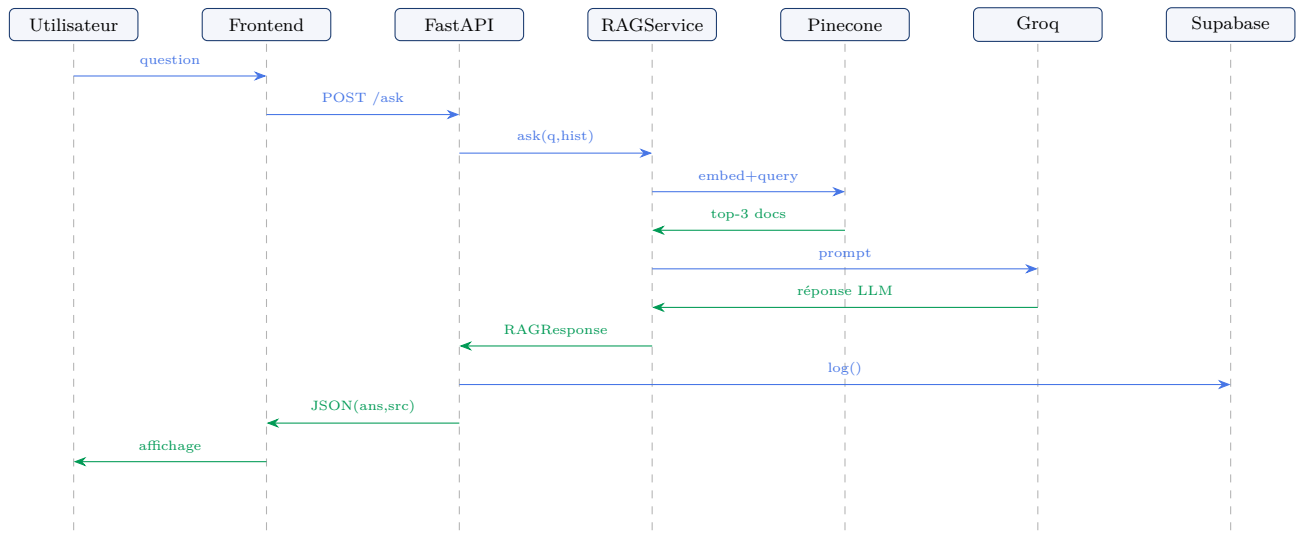


Figure 3.4 – Diagramme de séquence — Flux principal de traitement d’une question

3.5 Modèle de données

La base de données Supabase PostgreSQL contient une seule table `chat_logs` dont la structure est décrite dans le tableau 3.1.

Table 3.1 – Structure de la table `chat_logs`

Colonne	Type	Contrainte	Description
id	INTEGER	PRIMARY KEY, AUTO	Identifiant unique de l’interaction
timestamp	TIMESTAMP	DEFAULT now()	Date et heure de l’échange
question	TEXT	NOT NULL	Question posée par l’utilisateur
reponse	TEXT	NOT NULL	Réponse générée par le chatbot
sources	TEXT	NULLABLE	Sources citées (noms de fichiers, séparés par des virgules)

3.6 Description des composants

Table 3.2 – *Description des composants de l'application*

Fichier	Rôle	Responsabilités
<code>app/main.py</code>	Point d'entrée API	Endpoints REST, gestion du cycle de vie, service CORS
<code>app/rag.py</code>	Moteur RAG	Embeddings, retrieval Pinecone, génération LLM, formatage du contexte
<code>app/database.py</code>	Couche persistance	Connexion Supabase, ORM SQLAlchemy, journalisation
<code>scripts/load_vectors.py</code>	Indexation documentaire	Chargement PDF/TXT, chunking, vectorisation, upsert Pinecone
<code>index.html</code>	Interface utilisateur	SPA en HTML/CSS/JS, appels API, affichage des réponses et sources
<code>.env.example</code>	Configuration	Template des variables d'environnement sensibles
<code>requirements.txt</code>	Dépendances	Liste des paquets Python avec versions épinglées

Réalisation et développement

4.1 Environnement de développement

4.1.1 Environnement matériel

Table 4.1 – *Environnement matériel de développement*

Composant	Spécification
Processeur	Intel Core i5 / AMD Ryzen 5 (ou supérieur)
Mémoire RAM	8 Go minimum
Espace disque	5 Go (projet + environnement virtuel)
Connexion	Internet requis (APIs cloud : Groq, Pinecone, Supabase, Gemini)
Système d'exploitation	Windows 10/11, macOS ou Linux (Ubuntu 22.04+)

4.1.2 Environnement logiciel

Table 4.2 – *Environnement logiciel utilisé*

Outil	Version	Rôle
Python	3.11+	Langage principal du backend
pip	23.x	Gestionnaire de paquets
FastAPI	0.135.1	Framework web asynchrone
Uvicorn	0.42.0	Serveur ASGI
LangChain	1.2.12	Orchestration du pipeline RAG
langchain-groq	1.1.2	Intégration LLM Groq
langchain-google-genai	4.2.1	Embeddings Gemini
langchain-pinecone	0.2.13	Connecteur Pinecone
pinecone	7.3.0	Client Python Pinecone
pypdf	6.10.2	Extraction texte PDF
SQLAlchemy	2.0.48	ORM Python
psycopg2-binary	2.9.11	Driver PostgreSQL
pydantic	2.12.5	Validation des données
python-dotenv	1.2.2	Gestion des variables d'environnement
VS Code / PyCharm	–	Éditeur de code
Git	2.x	Contrôle de version

4.2 Technologies utilisées

4.2.1 Python et FastAPI

FastAPI est un framework web Python haute performance, basé sur **Starlette** et **Pydantic**. Il offre la génération automatique de documentation Swagger, la validation des requêtes via Pydantic et la compatibilité avec les opérations asynchrones. L'API expose trois endpoints :

- GET / : sert l'interface `index.html`.
- GET /health : retourne l'état du service RAG.
- POST /ask : reçoit une question et retourne une réponse RAG.

4.2.2 LangChain et le pipeline RAG

LangChain est un framework d'orchestration pour applications LLM. Dans ce projet, il est utilisé pour chaîner les composants suivants via l'opérateur pipe (`|`) :

```
1 self.chain = self.prompt | self.llm | StrOutputParser()
```

Listing 4.1 – Chaîne RAG définie dans *rag.py*

Le prompt système est rédigé en français et contient trois variables dynamiques : l'historique, le contexte documentaire et la question.

4.2.3 Pinecone — Base vectorielle

Pinecone est une base de données vectorielle hébergée dans le cloud. Le projet y stocke les vecteurs des chunks documentaires. Les chunks sont identifiés par leur empreinte MD5 pour éviter les doublons lors des réindexations.

Paramètres de chunking :

Table 4.3 – Paramètres du découpage textuel

Paramètre	Valeur
Taille du chunk	1 000 caractères
Chevauchement (overlap)	150 caractères
Stratégie	RecursiveCharacterTextSplitter
Taille des batchs upsert	100 vecteurs
Nombre de voisins (k)	3

4.2.4 Google Gemini Embeddings

Le modèle `models/gemini-embedding-001` de Google est utilisé pour transformer les chunks textuels en vecteurs denses. Ce modèle produit des embeddings de haute qualité pour le français.

4.2.5 Groq — LLM d'inférence

[Groq](#) est une plateforme d'inférence LLM à très faible latence. Le modèle `llama-3.3-70b-versatile` est utilisé avec les paramètres suivants :

Table 4.4 – Paramètres du LLM Groq

Paramètre	Valeur
Modèle	<code>llama-3.3-70b-versatile</code>
Température	0.3 (réponses factuelles)
Timeout	30 secondes
Type	Chat Completion

4.2.6 Supabase PostgreSQL

[Supabase](#) fournit une instance PostgreSQL managée dans le cloud. La connexion est établie via [SQLAlchemy](#) avec l'option `pool_pre_ping=True` pour la résilience des connexions.

4.3 Structure du projet

```

1  estfbs_assist/
2  |-- app/
3  |   |-- main.py           # Entrypoint FastAPI (136 lignes)
4  |   |-- rag.py            # Pipeline RAG (138 lignes)
5  |   +-- database.py       # Couche persistance (89 lignes)
6  |-- data/
7  |   |-- description_donnees_reglement_EST_FBS.pdf (98 Ko)
8  |   |-- CNPN_Diplome_Universitaire_de_Technologie.pdf (390 Ko)
9  |   +-- presentation.txt  (1,5 Ko)
10 |-- scripts/
11 |   +-- load_vectors.py    # Indexation (87 lignes)
12 |-- index.html            # SPA Frontend (1171 lignes)
13 |-- requirements.txt      # 15 dependances Python
14 |-- .env.example          # Template configuration
15 +-- README.md

```

Listing 4.2 – Arborescence du projet

4.4 Description des modules développés

4.4.1 Module API — app/main.py

Ce module constitue le point d'entrée de l'application. Il initialise le service RAG et la base de données au démarrage via le gestionnaire de contexte asynchrone `lifespan`, puis expose les endpoints REST. La validation des entrées est assurée par le modèle `Pydantic Question`.

```

1 @app.post("/ask")
2 def poser_question(q: Question) -> dict[str, Any]:
3     """Answer a student question and log the interaction."""
4     if rag_service is None:
5         raise HTTPException(status_code=503,
6                             detail="RAG service is not initialized.
7                                 ")
8     try:
9         question = q.normalized_text()
10        result = rag_service.ask(question, q.history)
11        log_interaction(question, result.answer, result.sources)
12        return {"answer": result.answer,
13               "reponse": result.answer,
14               "sources": result.sources}
15    except ValueError as exc:
16        raise HTTPException(status_code=400, detail=str(exc)) from
17        exc

```

Listing 4.3 – Endpoint principal *POST /ask*

4.4.2 Module RAG — app/rag.py

Ce module contient la classe `RAGService` qui encapsule l'intégralité du pipeline RAG. Son constructeur initialise les quatre composants principaux : embeddings, vector store, LLM et chaîne de traitement.

```

1 self.prompt = ChatPromptTemplate.from_template(
2     """Tu es l'assistant virtuel officiel de l'EST Fquih Ben Salah
3     .
4     Ton role est d'aider les etudiants en repondant a leurs questions.
5     Utilise uniquement le contexte fourni ci-dessous pour repondre.
6     Si la reponse ne se trouve pas dans le contexte, dis politement
7     que tu ne sais pas et conseille de contacter l'administration.
8
9     Historique de conversation:
10    {history}
11
12    Contexte extrait des documents:

```

```
12 {context}
13
14 Question de l'étudiant:
15 {question}
16
17 Reponse: ""
18 )
```

Listing 4.4 – *Prompt système du RAG en français*

4.4.3 Module Base de données — `app/database.py`

Ce module gère la persistance des interactions via SQLAlchemy. Il implémente un pattern singleton pour la gestion du moteur de connexion et de la session factory.

4.4.4 Script d'indexation — `scripts/load_vectors.py`

Ce script autonome prend en charge le chargement des documents dans Pinecone. Il lit les fichiers PDF et TXT du répertoire `data/`, les découpe en chunks, génère leurs embeddings et les insère par batch de 100 dans Pinecone. L'identification des chunks par empreinte MD5 assure l'idempotence des réindexations.

4.4.5 Frontend — `index.html`

L'interface utilisateur est une Single-Page Application (SPA) réalisée en HTML, CSS et JavaScript vanille (1 171 lignes). Ses caractéristiques principales sont :

- Design sombre à gradient bleu/violet avec variables CSS (`:root`).
- Sidebar de navigation avec 6 questions suggérées (admissions, formations, campus).
- Zone de chat avec indicateur de frappe animé (*typing dots*).
- Indicateur de statut temps réel (vert = en ligne / rouge = hors ligne).
- Affichage du nombre de segments vectorisés et du modèle LLM actif.
- Gestion de la touche Entrée pour l'envoi, auto-redimensionnement du textarea.
- Affichage des sources sous forme de *chips* colorés.
- Compatibilité multilingue affichée : FR, EN, AR.

4.5 Implémentation du chatbot

4.5.1 Flux de traitement d’une question

Voici le code JavaScript qui gère l’envoi d’une question et l’affichage de la réponse :

```

1  async function sendQuestion(rawQuestion) {
2      const question = rawQuestion.trim();
3      if (!question || state.waiting) return;
4
5      hideWelcome();
6      addMessage('user', question);
7      els.input.value = '';
8      setWaiting(true);
9      addTypingIndicator();
10
11     try {
12         const result = await postQuestion(question);
13         removeTypingIndicator();
14         addMessage('bot', result.answer, result.sources);
15         state.history.push({ user: question, bot: result.answer });
16         state.history = state.history.slice(-10);
17     } catch (error) {
18         removeTypingIndicator();
19         addMessage('bot', 'Impossible de joindre l API EST FBS...');
20     } finally {
21         setWaiting(false);
22         els.input.focus();
23     }
24 }

```

Listing 4.5 – Fonction principale d’envoi de question (*index.html*)

4.6 Gestion des données

4.6.1 Documents indexés

Table 4.5 – Documents du corpus documentaire

Fichier	Type	Taille	Contenu
presentation.txt	TXT	1,5 Ko	Présentation EST FBS, fi- lières, contacts
description_donnees_reglement_EST_FBS.pdf	PDF	98 Ko	Règlement inté- rieur, données administratives
CNPN_Diplôme Universitaire de Technologie.pdf	PDF	390 Ko	Programme national DUT (CNPN)

4.7 Difficultés rencontrées et solutions apportées

Table 4.6 – *Difficultés et solutions*

Difficulté	Solution apportée
Extraction de texte depuis des PDFs de faible qualité	Utilisation de <code>PyPDFLoader</code> de <code>LangChain-Community</code> avec <code>pypdf 6.x</code>
Déduplication lors des réindexations répétées	Identifiants de vecteurs basés sur l’empreinte MD5 du contenu des chunks
Gestion du contexte multi-tours	Transmission des 6 derniers échanges dans le prompt, formatés comme texte structuré
Compatibilité du champ question (question/text)	Modèle <code>Pydantic</code> avec deux champs optionnels et méthode <code>normalized_text()</code>
Initialisation asynchrone des services	Pattern <code>lifespan</code> de <code>FastAPI</code> pour l’initialisation au démarrage
Connexions DB instables	Option <code>pool_pre_ping=True</code> dans <code>SQLAlchemy</code>

Tests, limites et perspectives

5.1 Stratégie de test

Les tests ont été menés selon trois niveaux complémentaires :

1. **Tests unitaires manuels** : vérification de chaque composant isolément (chargement des documents, connexion aux APIs, validation Pydantic).
2. **Tests fonctionnels** : validation des scénarios principaux via l'interface et via des requêtes HTTP directes.
3. **Tests de robustesse** : envoi de questions hors périmètre, de questions vides et de requêtes malformées.

5.2 Tests fonctionnels

Table 5.1 – Résultats des tests fonctionnels

ID	Scénario	Résultat attendu	Statut
TF-01	Question sur les formations DUT	Liste des filières avec descriptions	✓
TF-02	Conditions d'admission	Critères d'admission selon le règlement	✓
TF-03	Localisation de l'école	Adresse complète et contact	✓
TF-04	Question hors périmètre	Message poli invitant à contacter l'administration	✓
TF-05	Question vide	Erreur HTTP 400	✓
TF-06	Endpoint <code>/health</code>	JSON avec statut, modèle, chunk_count	✓
TF-07	Question multi-tours	Cohérence contextuelle sur 3 échanges	✓
TF-08	Affichage des sources	Noms de fichiers sources affichés	✓
TF-09	Journalisation Supabase	Entrée créée dans <code>chat_logs</code>	✓
TF-10	Service indisponible (rag=None)	Erreur HTTP 503	✓

5.3 Analyse des performances

Table 5.2 – *Indicateurs de performance*

Métrique	Valeur mesurée	Commentaire
Chunks dans Pinecone (corpus initial)	≈ 29	3 documents indexés
Temps de retrieval Pinecone	< 0.5 s	API cloud
Temps d'inférence Groq	1–4 s	Latence Groq API
Temps total (question → réponse)	2–8 s	Dépend du réseau
Timeout configuré	30 s	<code>request_timeout</code>

5.4 Limites du système

Plusieurs limites ont été identifiées lors des phases de test :

- **Limite du corpus** : Les réponses sont strictement limitées aux documents indexés. Un document manquant ou mal extrait entraîne une réponse incomplète.
- **Qualité de l'extraction PDF** : L'extraction textuelle des PDFs scannés ou complexes (tableaux, colonnes) peut être imparfaite.
- **Configuration CORS restrictive** : Le middleware CORS ne permet que les requêtes depuis `localhost:3000`, ce qui bloque les déploiements en production sans modification.
- **Absence de tests automatisés** : Le projet ne contient pas de suite de tests unitaires ou d'intégration formelle (pytest).
- **Dépendance aux APIs tierces** : Le système requiert la disponibilité simultanée de quatre services cloud (Groq, Pinecone, Gemini, Supabase).
- **Pas d'authentification** : L'API est accessible sans authentification, ce qui peut poser des problèmes en production.
- **Stockage des sources** : Les sources sont stockées sous forme de chaîne texte dans la colonne `sources` de la table `chat_logs`, ce qui limite les requêtes analytiques.

5.5 Perspectives d'amélioration

Table 5.3 – *Pistes d'amélioration futures*

Priorité	Amélioration	Description
Haute	Enrichissement du corpus	Indexer l'ensemble du site web, les emplois du temps et les actualités
Haute	Déploiement production	Dockerisation + déploiement sur Railway, Render ou serveur USMS
Haute	Tests automatisés	Mise en place d'une suite pytest avec mocks des APIs tierces
Moyenne	Authentification	Ajout d'un système de tokens JWT pour sécuriser l'API
Moyenne	CORS dynamique	Configuration CORS basée sur des origines autorisées (env variable)
Moyenne	Tableau de bord	Interface d'administration pour visualiser les logs et les statistiques
Faible	Support arabe	Amélioration du support de la langue arabe pour les étudiants
Faible	Évaluation RAG	Métriques RAGAS (faithfulness, relevance) pour évaluer la qualité

Conclusion générale

Ce projet de fin d'études avait pour ambition de concevoir et développer un chatbot d'assistance intelligent pour l'École Supérieure de Technologie de Fkih Ben Salah, en réponse à un besoin réel d'amélioration de l'accès à l'information pour les étudiants et les candidats.

Le système développé repose sur l'architecture *Retrieval-Augmented Generation* (RAG), qui combine la puissance des grands modèles de langage avec la précision de la recherche vectorielle. Cette approche garantit que les réponses générées sont ancrées dans les documents officiels de l'établissement, assurant ainsi leur fiabilité et leur traçabilité.

L'ensemble des objectifs initiaux ont été atteints : le pipeline RAG est opérationnel, l'API FastAPI expose les fonctionnalités requises, l'interface web offre une expérience utilisateur moderne et intuitive, et chaque interaction est journalisée dans la base de données cloud Supabase.

D'un point de vue technique, ce projet a permis de mettre en pratique de nombreuses compétences acquises durant la formation DUT en Informatique Décisionnelle et Statistiques : programmation Python avancée, conception d'APIs REST, intégration de services cloud, manipulation de bases de données relationnelles et vectorielles, et développement frontend.

Les perspectives d'évolution sont nombreuses et prometteuses : enrichissement du corpus documentaire, déploiement en production sur l'infrastructure de l'USMS, ajout d'un mécanisme d'authentification et mise en place d'un tableau de bord analytique pour l'administration.

Ce projet illustre concrètement comment l'intelligence artificielle et le traitement du langage naturel peuvent être mobilisés au service des établissements d'enseignement supérieur pour améliorer l'expérience étudiante et optimiser les processus administratifs.

Bibliographie

1. **Lewis, P., Perez, E., et al.** (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020. <https://arxiv.org/abs/2005.11401>
2. **Documentation officielle FastAPI.** Tiangolo, S. <https://fastapi.tiangolo.com>
3. **Documentation LangChain.** LangChain, Inc. <https://python.langchain.com>
4. **Pinecone Documentation.** Pinecone Systems, Inc. <https://docs.pinecone.io>
5. **Groq Documentation.** Groq, Inc. <https://console.groq.com/docs>
6. **Google Gemini Embeddings.** Google DeepMind. <https://ai.google.dev/gemini-api/docs/embeddings>
7. **Supabase Documentation.** Supabase, Inc. <https://supabase.com/docs>
8. **SQLAlchemy Documentation.** <https://docs.sqlalchemy.org>
9. **Pydantic Documentation.** Pydantic Services Inc. <https://docs.pydantic.dev>
10. **Site officiel de l'EST Fkih Ben Salah.** Université Sultan Moulay Slimane. <https://estfbs.usms.ac.ma>
11. **CNPN DUT — Diplôme Universitaire de Technologie.** Cahier National des Normes Pédagogiques. Ministère de l'Enseignement Supérieur du Maroc.
12. **Vaswani, A., et al.** (2017). *Attention is All You Need*. NeurIPS 2017. <https://arxiv.org/abs/1706.03762>

Variables d'environnement

Le fichier `.env` (jamais commité) doit contenir les variables suivantes :

```
1 GOOGLE_API_KEY=your_google_ai_studio_api_key_here
2 GROQ_API_KEY=your_groq_api_key_here
3 GROQ_MODEL=llama-3.3-70b-versatile
4 PINECONE_API_KEY=your_pinecone_api_key_here
5 PINECONE_INDEX_NAME=your_pinecone_index_name_here
6 DATABASE_URL=your_supabase_postgres_connection_string_here
```

Listing A.1 – *Contenu du fichier `.env.example`*

Guide d'installation et de démarrage

```
1 # 1. Cloner le depot
2 git clone https://github.com/zorvex13/estfbs_assist.git
3 cd estfbs_assist
4
5 # 2. Creer et activer un environnement virtuel
6 python -m venv venv
7 source venv/bin/activate      # Linux/macOS
8 # venv\Scripts\activate      # Windows
9
10 # 3. Installer les dependances
11 pip install -r requirements.txt
12
13 # 4. Copier et remplir le fichier de configuration
14 cp .env.example .env
15 # Editer .env avec les cles API
16
17 # 5. Indexer les documents dans Pinecone
18 python scripts/load_vectors.py
19
20 # 6. Lancer le serveur FastAPI
21 uvicorn app.main:app --reload
22
23 # L'application est accessible a http://localhost:8000
```

Listing B.1 – *Commandes d'installation*

Endpoints de l'API — Documentation Swagger

FastAPI génère automatiquement une documentation Swagger accessible à l'adresse <http://localhost:8000/docs>. Le tableau C.1 récapitule les endpoints disponibles.

Table C.1 – *Endpoints de l'API FastAPI*

Méthode	Endpoint	Code	Description
GET	/	200	Retourne l'interface web HTML
GET	/health	200	État du service (ok, model, chunk_count)
POST	/ask	200	Traite une question et retourne une réponse RAG
POST	/ask	400	Question vide ou invalide
POST	/ask	503	Service RAG non initialisé
POST	/ask	500	Erreur interne du service IA

Diagrammes PlantUML

Les diagrammes suivants sont fournis en code PlantUML pour une intégration dans des outils de documentation.

```

1 @startuml
2 actor Utilisateur
3 participant "Frontend\n(index.html)" as FE
4 participant "FastAPI\n(main.py)" as API
5 participant "RAGService\n(rag.py)" as RAG
6 participant "Pinecone" as PC
7 participant "Groq LLM" as LLM
8 database "Supabase\nPostgreSQL" as DB
9
10 Utilisateur -> FE: Saisit une question
11 FE -> API: POST /ask {question, history}
12 API -> RAG: ask(question, history)
13 RAG -> PC: embed + similarity_search(k=3)
14 PC --> RAG: top-3 chunks
15 RAG -> LLM: prompt(history, context, question)
16 LLM --> RAG: reponse generee
17 RAG --> API: RAGResponse(answer, sources)
18 API -> DB: log_interaction()
19 API --> FE: JSON {answer, sources}
20 FE --> Utilisateur: Affiche reponse + sources
21 @enduml

```

Listing D.1 – *Diagramme de séquence PlantUML*

```

1 @startuml
2 class Question {
3     +question: str | None
4     +text: str | None
5     +history: list[dict]
6     +normalized_text(): str
7 }
8
9 class RAGService {
10     +embeddings: GoogleGenerativeAIEmbeddings
11     +vector_store: PineconeVectorStore
12     +retriever: VectorStoreRetriever

```

```
13  +llm: ChatGroq
14  +prompt: ChatPromptTemplate
15  +chain: Runnable
16  +ask(question, history): RAGResponse
17  -_format_docs(docs): str
18  -_format_history(history): str
19  }
20
21  class RAGResponse {
22    +answer: str
23    +sources: list[str]
24  }
25
26  class ChatLog {
27    +id: int
28    +timestamp: datetime
29    +question: str
30    +reponse: str
31    +sources: str
32  }
33
34  Question --> RAGService : utilise
35  RAGService --> RAGResponse : retourne
36  ChatLog ..|> RAGResponse : persiste
37  @enduml
```

Listing D.2 – *Diagramme de classes PlantUML*